

Builder

Le Pattern Builder

1. Définition

Le **Builder Pattern** est un patron de conception qui sépare la construction d'un objet complexe de sa représentation. Il permet de créer des objets étape par étape, rendant l'instanciation plus flexible et lisible.

Ce pattern s'avère particulièrement utile pour les objets ayant de nombreux attributs, certains étant optionnels ou ayant des valeurs par défaut. Au lieu d'utiliser plusieurs constructeurs avec différentes combinaisons de paramètres, le Builder offre une construction progressive et intuitive.

2. Structure du Pattern

Le **Builder Pattern** comprend quatre éléments principaux :

- **Le produit (Product)** : L'objet final créé par le builder.
- **Le constructeur (Builder)** : Une interface ou classe abstraite définissant les étapes de construction.
- **Le builder concret (ConcreteBuilder)** : Une classe qui implémente la construction étape par étape.
- **Le directeur (Director) [Optionnel]** : Une classe orchestrant la création en appelant les méthodes du builder dans un ordre précis.

3. Problème du code sans Builder

Prenons l'exemple d'une classe `Computer` avec différents attributs : CPU, RAM, stockage, GPU et système d'exploitation.

Une implémentation classique utilise plusieurs constructeurs :

```
public class Computer {  
    private final String cpu;  
    private final String ram;  
    private final String storage;  
    private final boolean hasGpu;  
    private final String os;
```

```

// Constructeur avec tous les paramètres
public Computer(String cpu, String ram, String storage, boolean hasGpu,
String os) {
    this.cpu = cpu;
    this.ram = ram;
    this.storage = storage;
    this.hasGpu = hasGpu;
    this.os = os;
}

// Constructeur avec OS par défaut
public Computer(String cpu, String ram, String storage, boolean hasGpu)
{
    this(cpu, ram, storage, hasGpu, "Windows 10");
}

// Constructeur avec OS et GPU par défaut
public Computer(String cpu, String ram, String storage) {
    this(cpu, ram, storage, false, "Windows 10");
}
}

```

Problèmes avec cette approche :

- Difficulté à distinguer les différents constructeurs.
- Nécessité de respecter strictement l'ordre des paramètres.
- Prolifération des constructeurs lors de l'ajout de nouveaux attributs.

4. Solution avec le Builder Pattern

Une approche plus élégante consiste à utiliser une classe `Builder` permettant de définir les attributs individuellement :

```

public class Computer {
    private final String cpu;
    private final String ram;
    private final String storage;
    private final boolean hasGpu;

```

```

private final String os;

private Computer(Builder builder) {
    this.cpu = builder.cpu;
    this.ram = builder.ram;
    this.storage = builder.storage;
    this.hasGpu = builder.hasGpu;
    this.os = builder.os;
}

public static class Builder {
    private String cpu;
    private String ram;
    private String storage;
    private boolean hasGpu = false;
    private String os = "Windows 10";

    public Builder setCpu(String cpu) {
        this.cpu = cpu;
        return this;
    }
    public Builder setRam(String ram) {
        this.ram = ram;
        return this;
    }
    public Builder setStorage(String storage) {
        this.storage = storage;
        return this;
    }
    public Builder setGpu(boolean hasGpu) {
        this.hasGpu = hasGpu;
        return this;
    }
    public Builder setOs(String os) {
        this.os = os;
        return this;
    }
    public Computer build() {

```

```

        return new Computer(this);
    }
}
}

```

5. Utilisation du Builder

Cette approche rend la création d'objets plus intuitive et flexible :

```

Computer gamingComputer = new Computer.Builder()
    .setCpu("Intel i9")
    .setRam("64GB")
    .setStorage("2TB SSD")
    .setGpu(true)
    .setOs("Windows 11")
    .build();

```

```

Computer officeComputer = new Computer.Builder()
    .setCpu("Intel i5")
    .setRam("16GB")
    .setStorage("512GB SSD")
    .build();

```

6. Avantages du Builder Pattern

✅ **Lisibilité améliorée** : Plus besoin de mémoriser l'ordre des paramètres.



Flexibilité accrue : Ajout facile de nouveaux attributs.



Facilité d'utilisation : Configuration uniquement des paramètres nécessaires.



Meilleure maintenabilité : Évite la multiplication des constructeurs.

7. Différences avec l'implémentation GoF (Gang of Four)

Notre exemple présente une version simplifiée du **Builder Pattern**.

L'implémentation GoF propose :

- **Un même Builder peut créer plusieurs types de produits** (ex. : PC de bureau, laptop, etc.).

- **Un Director peut être utilisé** pour orchestrer la construction en appelant le Builder selon une séquence définie.

Voici un exemple avec plusieurs types de produits :

```
public class GamingComputerBuilder extends Computer.Builder {  
    public GamingComputerBuilder() {  
        setGpu(true);  
        setOs("Windows 11");  
    }  
}
```

```
Computer gamingComputer = new GamingComputerBuilder()  
    .setCpu("Intel i9")  
    .setRam("64GB")  
    .setStorage("2TB SSD")  
    .build();
```

8. Conclusion

Le **Builder Pattern** offre une solution élégante pour construire des objets complexes par étapes, apportant clarté et flexibilité. Il est particulièrement adapté lorsque :

- L'objet comporte plusieurs attributs optionnels.
- La complexité des constructeurs doit être évitée.
- La création peut être décomposée en étapes distinctes.

En définitive, il représente une alternative efficace aux constructeurs traditionnels, améliorant significativement la lisibilité et la maintenabilité du code.